Databases for Data Analytics

Task 8 - Performance | Yaron Besser

Question 1 - Original Query Execution Plan

Task: *Compute the execution plan and time of the original query.*

Row count: 764,416 rows

SQL Query:

```
-- Comedies pairs of the same director (original query)
SELECT *
FROM movies_genres AS fmg
JOIN movies AS fm
    ON fmg.movie_id = fm.id
JOIN imdb_ijs.movies_directors AS fmd
    ON fm.id = fmd.movie_id
JOIN imdb_ijs.movies_directors AS smd
    ON fmd.director_id = smd.director_id
JOIN imdb_ijs.movies AS sm
    ON smd.movie_id = sm.id
JOIN imdb_ijs.directors AS d
    ON fmd.director_id = d.id
JOIN movies_genres AS smg
    ON sm.id = smg.movie_id
WHERE fmd.movie_id != smd.movie_id
    AND fmd.movie_id > smd.movie_id
    AND fmg.genre = 'Comedy'
    AND smg.genre = 'Comedy'
ORDER BY d.first_name, d.last_name, fm.name, sm.name;
```

Execution plan (EXPLAIN):

id	select_ty...	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	fmg	NULL	index	PRIMARY,movies_genres_movie_id	movies_genres_movie_id	4	NULL	394949	10.00	Using where; Using index; Using temporary; Usi...
1	SIMPLE	fm	NULL	eq_ref	PRIMARY	PRIMARY	4	imdb_ijs.fmg.movie_id	1	100.00	NULL
1	SIMPLE	fmd	NULL	ref	PRIMARY,movies_directors_director_id,movies...	movies_directors_movie_id	4	imdb_ijs.fmg.movie_id	1	100.00	Using index
1	SIMPLE	d	NULL	eq_ref	PRIMARY	PRIMARY	4	imdb_ijs.fmd.director_id	1	100.00	NULL
1	SIMPLE	smd	NULL	ref	PRIMARY,movies_directors_director_id,movies...	movies_directors_director_id	4	imdb_ijs.fmd.director_id	4	30.00	Using where; Using index
1	SIMPLE	sm	NULL	eq_ref	PRIMARY	PRIMARY	4	imdb_ijs.smd.movie_id	1	100.00	NULL
1	SIMPLE	smg	NULL	eq_ref	PRIMARY,movies_genres_movie_id	PRIMARY	306	imdb_ijs.smd.movie_id,const	1	100.00	Using index

Timing:

	Time	Action	Response	Duration / Fetch Time
1	17:32:41	SELECT * FROM movies_genres AS fmg JOIN movies AS fm ON...	764416 row(s) returned	11.617 sec / 0.347 sec

Explanation:

The query scans all 394,949 rows in movies_genres and throws away ~355,000 that are not comedies — this full scan is the main bottleneck. All other joins use indexes and are fast. Total: 764,416 rows, 11.617 seconds.

Question 2 - Query Optimization Steps

Task: *Improve the query in steps (at least 3). Per step: explain the intuition, implement the change, compute the new execution plan and time, and explain the outcome.*

Assumptions: The optimized queries use SELECT * to match the original query's output. Since the task asks for comedy pairs rather than specific columns, this is an assumption — in practice, explicit columns would be preferred. Similarly, ORDER BY clauses are added for readability and consistent output comparison across steps, but were not required by the task.

Step 1 - Remove Unnecessary Table Joins

Intuition:

The original query joins 7 tables but only needs 4 — movies and directors are joined solely for display names. Removing them reduces lookups across 764,416 rows. md1.movie_id != md2.movie_id was also removed: it is redundant since md1.movie_id > md2.movie_id already guarantees the two IDs differ.

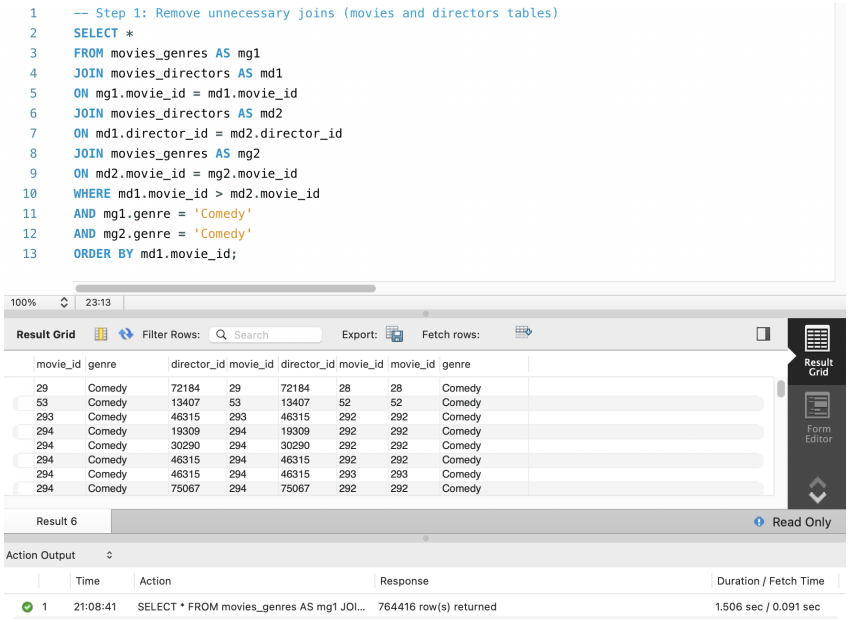
SQL Query:

```
-- Step 1: Remove unnecessary joins (movies and directors tables)
-- Also removed redundant != condition: md1.movie_id > md2.movie_id already implies !=
SELECT *
FROM movies_genres AS mg1
JOIN movies_directors AS md1
    ON mg1.movie_id = md1.movie_id
JOIN movies_directors AS md2
    ON md1.director_id = md2.director_id
JOIN movies_genres AS mg2
    ON md2.movie_id = mg2.movie_id
WHERE md1.movie_id > md2.movie_id
    AND mg1.genre = 'Comedy'
    AND mg2.genre = 'Comedy'
ORDER BY md1.movie_id;
```

Execution plan (EXPLAIN):

id	select_ty...	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	mg1	NULL	index	PRIMARY,movies_genres_movie_id	movies_genres_movie_id	4	NULL	394949	10.00	Using where; Using index; Using temporary; Usi...
1	SIMPLE	md1	NULL	ref	PRIMARY,movies_directors_director_id,movies...	movies_directors_movie_id	4	imdb_ijs.mg1.movie_id	1	100.00	Using index
1	SIMPLE	md2	NULL	ref	PRIMARY,movies_directors_director_id,movies...	movies_directors_director_id	4	imdb_ijs.md1.director_id	4	33.33	Using where; Using index
1	SIMPLE	mg2	NULL	eq_ref	PRIMARY,movies_genres_movie_id	PRIMARY	306	imdb_ijs.md2.movie_id,const	1	100.00	Using index

Result and timing:



Outcome:

11.617 sec → 1.506 sec, about 8x faster. EXPLAIN shows 4 tables instead of 7. movies_genres still scans 394,949 rows, so the Comedy filter bottleneck remains.

Step 2 - Pre-filter Comedies Using CREATE TABLE

Intuition:

movies_genres still scans 394,949 rows to find comedies. Physically writing just the comedy rows to a new table first means the self-join runs on ~54K rows instead of ~395K.

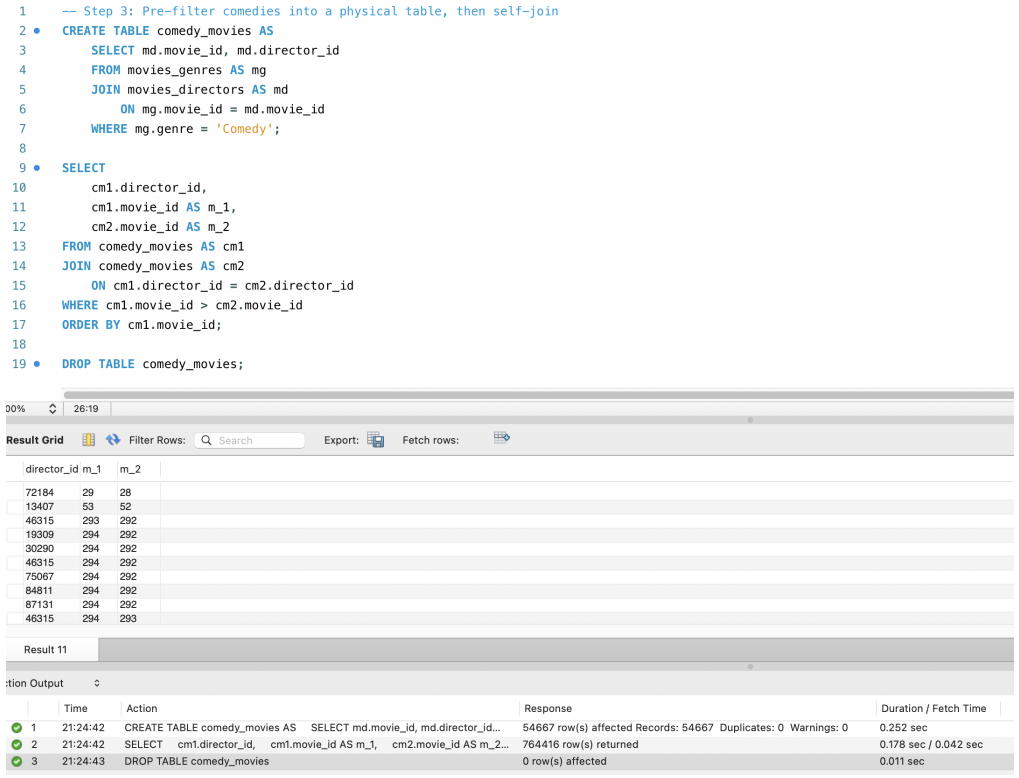
SQL Query:

```
-- Step 2: Pre-filter comedies into a physical table, then self-join
CREATE TABLE comedy_movies AS
    SELECT md.movie_id, md.director_id
    FROM movies_genres AS mg
    JOIN movies_directors AS md
        ON mg.movie_id = md.movie_id
    WHERE mg.genre = 'Comedy';

SELECT
    cm1.director_id,
    cm1.movie_id AS m_1,
    cm2.movie_id AS m_2
FROM comedy_movies AS cm1
JOIN comedy_movies AS cm2
    ON cm1.director_id = cm2.director_id
WHERE cm1.movie_id > cm2.movie_id
ORDER BY cm1.movie_id;

DROP TABLE comedy_movies;
```

Execution plan (EXPLAIN):

id	select_ty...	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	cm1	NULL	ALL	NULL	NULL	NULL	NULL	53725	100.00	Using temporary; Using filesort
1	SIMPLE	cm2	NULL	ALL	NULL	NULL	NULL	NULL	53725	3.33	Using where; Using join buffer (hash join)

Result and timing:



Outcome:

CREATE TABLE: 0.252 sec to build 54,667 rows. SELECT: 0.178 sec — down from 1.506 sec. EXPLAIN shows both sides scanning only 53,725 rows instead of 394,949.

Step 3 - Add Index on Genre + CREATE TABLE

Intuition:

Step 2 still filters 394,949 rows when building the comedy table. Adding an index on genre should let the database jump directly to comedy rows, making the CREATE TABLE step faster.

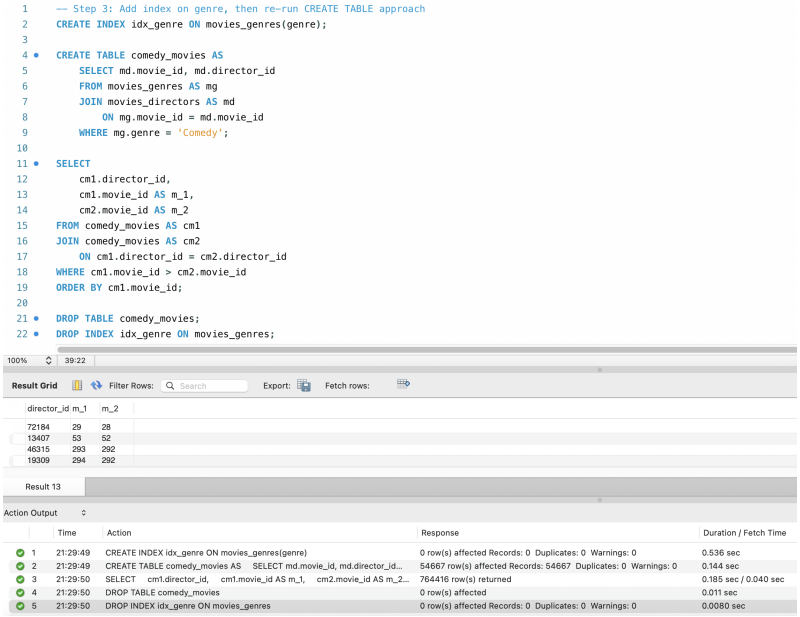
SQL Query:

```
-- Step 3: Add index on genre, then re-run CREATE TABLE approach
CREATE INDEX idx_genre ON movies_genres(genre);

CREATE TABLE comedy_movies AS
    SELECT md.movie_id, md.director_id
    FROM movies_genres AS mg
    JOIN movies_directors AS md
        ON mg.movie_id = md.movie_id
    WHERE mg.genre = 'Comedy';

SELECT
    cm1.director_id,
    cm1.movie_id AS m_1,
    cm2.movie_id AS m_2
FROM comedy_movies AS cm1
JOIN comedy_movies AS cm2
    ON cm1.director_id = cm2.director_id
WHERE cm1.movie_id > cm2.movie_id
ORDER BY cm1.movie_id;

DROP TABLE comedy_movies;
DROP INDEX idx_genre ON movies_genres;
```

Execution plan (EXPLAIN):

id	select_ty...	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	cm1	NULL	ALL	NULL	NULL	NULL	NULL	53200	100.00	Using temporary; Using filesort
1	SIMPLE	cm2	NULL	ALL	NULL	NULL	NULL	NULL	53200	3.33	Using where; Using join buffer (hash join)

Result and timing:



Outcome:

The index made CREATE TABLE faster: 0.144 sec vs 0.252 sec in step 2 — the genre filter now runs on an index instead of a full scan. The SELECT stayed similar at 0.185 sec since comedy_movies itself has no index on director_id. Total pipeline improved slightly but the self-join remains the limiting factor.

Question 2, Part 2 - Real World Commit Improvements

Task: *Answer the 'Real world commit improvements' question and add solutions for 3 performance queries.*

Query 1 - NOT IN Subquery

Original:

```
SELECT w.id
FROM georef.way AS w
WHERE w.id NOT IN (SELECT way_id FROM georef.fusion_ways);
```

Problem: The subquery runs once per row in the way table — extremely slow on large tables. Also, any NULL in way_id causes the entire result to return empty.

Improved:

```
SELECT w.id
FROM georef.way AS w
LEFT JOIN georef.fusion_ways AS fw
    ON w.id = fw.way_id
WHERE fw.way_id IS NULL;
```

Why it helps: The JOIN runs once across both tables. Unmatched rows get NULL in fw.way_id, which the WHERE clause filters for. Faster and handles NULLs correctly.

Query 2 - Unnecessary DISTINCT

Original:

```
SELECT DISTINCT customer_id, order_date
FROM orders;
```

Problem: DISTINCT forces a full sort and deduplication of all results before returning rows. Expensive on large tables.

Improved:

```
SELECT customer_id, order_date
FROM orders
GROUP BY customer_id, order_date;
```

Why it helps: GROUP BY can use a composite index on (customer_id, order_date) to read unique combinations directly, skipping the full sort.

Query 3 - DISTINCT Inside INSERT SELECT

Original:

```
INSERT INTO dbo.d_arvokyselykerta (
    SELECT DISTINCT
        -1, [source], koodi, nimi,
        koodi, nimi, nimi_sv, nimi_en, nimi, -1
    FROM ANTERO_SA.dbo.sa_koodistot
    WHERE koodisto = 'vipunenmeta'
        AND koodi = '-1'
);
```

Problem: DISTINCT forces a full sort and deduplication before every insert. There is also no index on the filter columns. *Note: 'duplicates are unlikely' is not a valid reason to remove DISTINCT — that is how data gets corrupted. The correct justification is that schema constraints should guarantee uniqueness, making DISTINCT redundant by design.*

Improved:

```
CREATE INDEX idx_koodisto_koodi
ON ANTERO_SA.dbo.sa_koodistot(koodisto, koodi);

INSERT INTO dbo.d_arvokyselykerta
SELECT
    -1, [source], koodi, nimi,
    koodi, nimi, nimi_sv, nimi_en, nimi, -1
FROM ANTERO_SA.dbo.sa_koodistot
WHERE koodisto = 'vipunenmeta'
    AND koodi = '-1';
```

Why it helps: Removing DISTINCT eliminates the sort and deduplication step. The index lets the database jump directly to matching rows instead of scanning the full table.